

TASK 5 REPORT

COS30018

INTELLIGENT SYSTEMS

Halim Vlahos (104015943)

CREATE MULTISTEP MULTIVARIATE SUMMARY

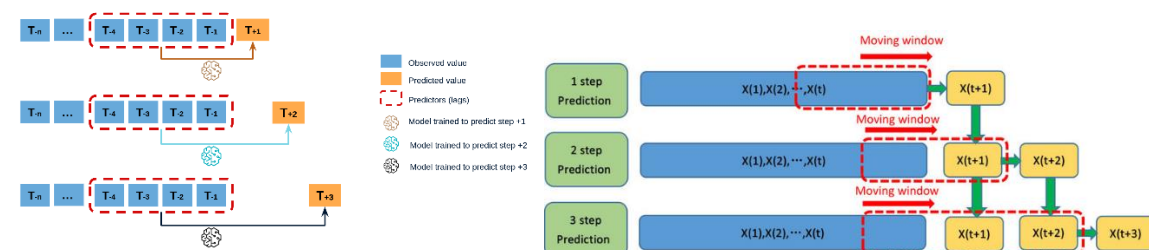
Multistep

```
296
297 def create_multistep_sequences(data, window_size, steps_ahead):
298     """
299     Creates input output pairs for multistep predictions
300     Input sequences contain data of past window size days
301     Output sequences contain data of next steps ahead
302     :param data: gets stock data
303     :param window_size: window size for past/input prices.
304     :param steps_ahead: prediction days (k)
305     :return: input and output sequence arrays.
306     """
307     X, y = [], []
308     for i in range(len(data) - window_size - steps_ahead):
309         #append input sequences of window size
310         X.append(data[i:i+window_size])
311         #append output sequences
312         y.append(data[i+window_size:i+window_size+steps_ahead])
313     return np.array(X), np.array(y)
314
315
316
```

The first objective was to “Implement a function that solve the multistep prediction problem to allow the prediction to be made for a sequence of closing prices of k days into the future.”

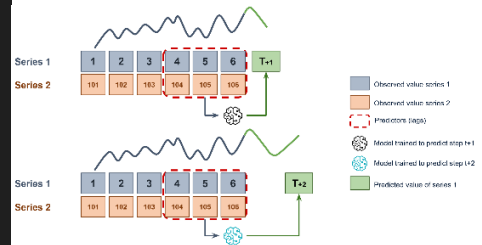
The `create_multistep_sequences` function creates input output pairs by using a moving window of `window_size` as values for input, and pairing with the output value of `steps_ahead`. The function uses a loop through the data to append the input and output sequence pairs.

These pairs prepare training data so that the model can learn to predict multiple days in the future.



Multivariate

```
317 def create_multivariate_sequences(data, window_size):
318     """
319     Creates input output pairs for multivariate prediction.
320     Input sequences contain the multivariate (open, high, low, close etc) data of the past window size days
321     Output sequence contains closing price of the next day.
322
323     :param data: multivariate data with columns ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
324     :param window_size: the number of past days to use as input
325     :return: X (input sequences), y (output target - closing price).
326     """
327     X, y = [], []
328
329     # get multivariate features and close price from data
330     feature_columns = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
331     target_column = 'Close'
332
333     # prepare input sequences and target (closing price)
334     for i in range(window_size, len(data)):
335         # input sequence: all features for the last window size days
336         X.append(data[feature_columns].iloc[i - window_size:i].values)
337         # target sequence: closing price for the next day
338         y.append(data[target_column].iloc[i])
339
340     return np.array(X), np.array(y)
```



The multivariate issue had to allow for other data feature columns (Open, High, Low, Close etc) to be taken into consideration, as opposed to basing all the training data on the Close values.

“Implement a function that solve the simple multivariate prediction problem to that takes into account the other features for the same company (including opening price, highest price, lowest price, closing price, adjusted closing price, trading volume) as the input for predicting the closing price of the company for a specified day in the future.”

This was done by again creating input output pairs, this time having the input sequences include all the features of the past window size days, and the output pair be the closing price (Close value) of the next day.

The pairs are used to prepare the training data for the model to use and learn how to predict the Close value based on all the feature values (highest price, lowest price etc).

Multistep-Multivariate

```
344 def create_multistep_multivariate_sequences(data, window_size, steps_ahead):
345     """
346     Creates input output pairs for multistep, multivariate prediction.
347     Input sequences contain the multivariate (open, high, low, close, etc.) data of past window size days.
348     Output sequences contain closing prices for the next steps ahead days.
349
350     :param data: multivariate data with columns ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
351     :param window_size: the number of past days to use as input
352     :param steps_ahead: number of future days to predict (k)
353     :return: X (input sequences), y (target - future closing prices).
354     """
355     X, y = [], []
356
357     # multivariate feature columns and the target column (Close price)
358     feature_columns = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
359     target_column = 'Close'
360
361     # prepare input sequences and output target (closing prices for steps ahead days)
362     for i in range(len(data) - window_size - steps_ahead):
363         # input sequence: multivariate data for the past window size days
364         X.append(data[feature_columns].iloc[i:i + window_size].values)
365
366         # target sequence: closing prices for the next steps ahead days
367         y.append(data[target_column].iloc[i + window_size:i + window_size + steps_ahead].values)
368
369     return np.array(X), np.array(y)
```

The create_multistep_multivariate_sequences function combined the two concepts to create input output pairs for training data, this time the input including all the necessary features, and the output being steps_ahead values. The model uses these pairs to learn from and make a days-ahead prediction based on multiple features.